STUDENT HANDOUT · DAY 2

Network, Port and Vulnerability Scanning

Digital Forensics & Cyber Security Bootcamp · Atomcamp

Trainer: Waseem Abbas Khan — Senior Cybersecurity Specialist, CEI Master Trainer

Purpose of this document

This handout records and expands everything covered in the Day 2 session. The session moved quickly and covered a large amount of command syntax; what follows consolidates all of it into one reference you can work from independently.

The material follows the order in which the work was actually performed: establishing a controlled laboratory, discovering live hosts, enumerating open ports and services, applying scripts, and finally converting a service version into a documented, scored and referenced vulnerability finding.

SCOPE Every command in this handout was executed against Metasploitable 2 at 192.168.50.129 from the course Kali virtual machine, on a NAT network. No command was run against any system outside that laboratory. That constraint is not a teaching convenience; it is the single most important professional habit you will carry from this course into employment.

Learning outcomes

By the end of this material you should be able to do each of the following without reference:

#	Outcome	Evidence produced
1	Discover live hosts on a subnet using three independent tools and explain when each fails	nmap -sn, netdiscover and ipscan output compared
2	Select the appropriate port-scanning form for a given objective, and justify the choice	Default, full-range, targeted and range scans
3	Explain precisely what -O, -sV, -sU, -A and -sC each do, and what each costs	Output for each flag against the target
4	Choose a timing template deliberately and articulate the trade-off	T0 versus T4 versus T5 results and timings
5	Describe what fragmentation and decoy scanning attempt, and where they fail today	nmap -f and nmap -D output plus packet capture
6	Run NSE scripts by category and by name, having first established what the script does	FTP, HTTP and SMB script output
7	Resolve a service banner to a CVE and read the CVSS vector metric by metric	CVE-2011-2523 fully documented
8	Classify the weakness using CWE and explain why that classification drives remediation	CWE-78 mapped to the finding
9	Write a finding that a client can act on, including a reproducible proof of concept	One complete finding to the report template

1. Laboratory Set-up

1.1 The environment

Role	Operating system	Address	Network
Attacker	Kali Linux	192.168.50.x	NAT
Target	Metasploitable 2 (Ubuntu-based Linux)	192.168.50.129	NAT
Target source	vulnhub.com/entry/metasploitable-2,29	—	—

Metasploitable 2 is a Linux virtual machine that has been deliberately built with dozens of vulnerable services. It exists so that students and professionals can practise exploitation and enumeration without harming a real system. Because it is intentionally insecure, it must never be connected to a network you do not control. The NAT configuration used here keeps both machines reachable to each other through the host, while preventing other devices on your home or office network from reaching the target.

WHY NAT, AND WHY IT MATTERS NAT is acceptable for this exercise because the target cannot be reached from outside your host. If you instead use a bridged adapter, the intentionally vulnerable machine becomes visible to every device on your local network, and it will be found. Somebody scans every network continuously. This is the first scope decision you make in any lab, and it is the one most often made badly.

1.2 The three scanning disciplines

The session covered three distinct activities. They are often discussed as if they were one thing, but they answer different questions and produce different deliverables. Understanding the boundary between them is what separates a structured assessment from aimless scanning.

Discipline	The question it answers	Typical tools	Deliverable
Network scanning	Which hosts are alive on this network?	nmap -sn, netdiscover, ipscan	A host inventory
Port scanning	Which ports are open, and what service runs on each?	nmap -p, -sS, -sU, -sV, -O	A service inventory
Vulnerability scanning	Which of those services are weak, and how weak?	NSE, CVE research, Nessus	Scored, validated findings

WHY THE ORDER IS FIXED The order is not optional, and it is not a formality. A port scan without a host list is scanning blindly. A vulnerability scan without a service inventory is guessing which services to examine. A port scan without a vulnerability review tells the client that a door is open but not what is behind it. Each stage narrows the next.

1.3 Network discovery with three tools

nmap host discovery

```
nmap -sn 192.168.50.0/24
```

EXPECTED OUTPUT

```
Nmap scan report for 192.168.50.1
Host is up (0.00042s latency).
MAC Address: 0A:00:27:00:00:00 (VirtualBox)

Nmap scan report for 192.168.50.129
Host is up (0.00089s latency).
MAC Address: 08:00:27:AA:BB:CC (VirtualBox)

Nmap done: 256 IP addresses (2 hosts up) scanned in 3.11 seconds
```

The -sn flag instructs nmap to perform host discovery only and to perform no port scanning. On a directly connected subnet nmap uses ARP requests for this purpose, because ARP cannot be filtered by a host firewall — a device must answer ARP in order to function on the network at all. This is why local discovery is markedly more reliable than remote discovery.

The MAC address in the output provides two pieces of information. First, the vendor string (VirtualBox) confirms that the host is a virtual machine, which is itself an observation about the environment rather than about the target. Second, the MAC address is the only identifier that is genuinely attributable to a physical or virtual network interface; IP addresses are borrowed and reassigned, MAC addresses are not.

netdiscover

```
sudo netdiscover -r 192.168.50.0/24 -P      # passive: prints results, then stops
sudo netdiscover -r 192.168.50.0/24         # interactive: press Ctrl-C to stop
```

EXPECTED OUTPUT

```
 Currently scanning: Finished!   |   Screen: 1
 1 Captured ARP Req/Rep packets, from 1 hosts.

   IP            At MAC Address     Count     Len  MAC Vendor / Hostname
 192.168.50.1    0a:00:27:00:00:00      2      84  PCS Systemtechnik GmbH
 192.168.50.129  08:00:27:aa:bb:cc      1      60  PCS Systemtechnik GmbH
```

netdiscover operates entirely at the ARP layer. The -r flag specifies the address range, and -P requests passive mode, in which the tool only listens to existing ARP traffic rather than generating its own. Passive mode is valuable because it produces no traffic of your own, which means the target has no record of your activity. The limitation is equally clear: it only sees ARP broadcast traffic, so on a quiet network it may find nothing at all, and it cannot operate outside the local subnet.

ipscan (graphical alternative)

A graphical scanner such as Angry IP Scanner provides a result table that is easy to screenshot and therefore easy to attach to a report. It is a reasonable choice when the deliverable requires screenshots. It does not offer the granular control that nmap's flags provide, and for that reason it should be a supplement rather than a replacement.

THE PRINCIPLE OF TWO SOURCES Using three tools is not redundancy for its own sake. Different tools use different discovery mechanisms, and therefore fail in different circumstances. When two independent methods agree, the result is defensible. When they disagree, the disagreement itself is informative — it usually means a filtering rule exists somewhere. State the agreement or the disagreement explicitly in your report.

2. Port Scanning

This section documents every port-scanning form used in the session. For each one, the command, the behaviour, and the circumstances in which it is the correct choice.

2.1 Establishing the target variable

```
TARGET=192.168.50.129
```

EXPECTED OUTPUT
(no output — shell variable assignment produces no output)

Assigning a value to a shell variable is silent by definition. The value is not confirmed until you print it, and printing it correctly requires the dollar sign:

```
echo TARGET      # prints the literal word TARGET
echo $TARGET     # prints the value held by the variable
```

EXPECTED OUTPUT
```
TARGET
192.168.50.129
```

A LESSON WORTH KEEPING Both of these commands were run during the session, and the difference between them is the most common shell error in this entire discipline. Omitting the dollar sign prints the variable name. Worse, if the variable has not been set at all, echo $TARGET prints an empty line and returns success — there is no error message to warn you. A scan command containing an unset variable may therefore run with no target whatsoever, and you will not be told. Always confirm the variable before you use it in a command.

2.2 Port selection forms

Command	Ports scanned	Appropriate when	Typical duration
nmap $TARGET	The top 1,000 TCP ports by frequency	First orientation on a new host	Seconds
nmap -p- $TARGET	All 65,535 TCP ports	You need completeness	Minutes
nmap -p0-65535 $TARGET	All 65,535 TCP ports	Identical to -p-; older notation	Minutes
nmap -p21 $TARGET	Port 21 only	Confirming a specific service	Under a second
nmap -p21,22,25,80 $TARGET	Exactly the listed ports	Checking a known set of services	Seconds
nmap -p21-100 $TARGET	The inclusive range 21–100	Investigating a suspected service block	Seconds

```
nmap $TARGET
nmap -p- $TARGET
nmap -p21 $TARGET
nmap -p21,22,25,80 $TARGET
nmap -p21-100 $TARGET
```

EXPECTED OUTPUT
```
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
80/tcp   open  http
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
8180/tcp open  unknown
```

THE PRACTICAL CONSEQUENCE OF -p- The default scan examines the 1,000 most commonly used TCP ports. It does not examine all ports. On this particular target, the backdoor shell from the FTP compromise listens on port 6200, which is not included in the default set. A student who scans only the defaults will conclude that nothing interesting is present on that host, and will be wrong. Whenever completeness matters — and in a vulnerability assessment it usually does — use -p-.

2.3 Operating system detection

```
sudo nmap -O $TARGET
```

EXPECTED OUTPUT

```
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
```

The -O flag performs TCP/IP stack fingerprinting: nmap sends carefully constructed packets and compares the responses against a database of known stack behaviour. It requires root privileges because it needs to craft raw packets.

Note carefully that the result is a range — Linux 2.6.9 through 2.6.33 — not a specific version. Fingerprinting infers the stack from its behaviour, and many kernel versions behave identically. In a report, write that the fingerprint is consistent with a given family; do not assert a version the tool did not establish.

2.4 UDP scanning

```
nmap -sU $TARGET
sudo nmap -sU --top-ports 20 $TARGET    # faster, same purpose
```

EXPECTED OUTPUT

```
PORT     STATE         SERVICE
53/udp   open          domain
111/udp  open          rpcbind
137/udp  open|filtered netbios-ns
```

UDP scanning is inherently slower and less certain than TCP scanning, because UDP provides no acknowledgement mechanism. A host that receives a UDP datagram for a closed port is expected to reply with an ICMP port-unreachable message, but many hosts and firewalls do not, and open ports that do not speak first produce no reply at all.

The open|filtered state is the honest expression of this uncertainty: nmap received no response, and therefore cannot distinguish between a closed port that was silently dropped and an open port that had nothing to say. Report it as unknown. Recording it as open is a factual error, and it will be challenged by anyone who reads the raw output.

2.5 Version detection

```
nmap -sV $TARGET
```

EXPECTED OUTPUT

```
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
445/tcp  open  netbios-ssn Samba smbd 3.3.2-SVN
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
```

THE MOST IMPORTANT FLAG OF THE SESSION This output is the pivot point of the entire day. Without version detection the result is 'port 21 open, service ftp', which supports no finding at all. With version detection it becomes 'vsftpd 2.3.4', and a version number is precisely what is required to consult a vulnerability database. Version detection is the step that converts port scanning into vulnerability assessment, and it is the single most valuable flag available to you.

2.6 Timing templates

Template	Name	Behaviour	Detectability	When to use
T0	Paranoid	One port at a time, long delays	Very slow to trigger thresholds	IDS evasion study, with authorisation only
T1	Sneaky	Slow, less extreme than T0	Low	The same objective, less waiting
T2	Polite	Reduced bandwidth and concurrency	Moderate	Fragile or production systems

T3	Normal	The default	Moderate	Whenever you have no reason to choose otherwise
T4	Aggressive	Assumes a fast, reliable network	Easy to detect	Your own laboratory
T5	Insane	Maximum speed, assumes a very fast network	Obvious	Only where missed results are acceptable

```
nmap -T0 -p 21,22 $TARGET     # observe the elapsed time
nmap -T4 -p 21,22 $TARGET
nmap -T5 -p 21,22 $TARGET
```

REFINING A COMMON STATEMENT It is worth refining the description given in the session. T0 is frequently described as 'undetectable'. It is not. T0 is slow enough that a request arrives well below the rate at which signature-based thresholds are typically configured to alert, and it is therefore less likely to be noticed as it happens. A monitoring system that correlates activity over a long window will still see it, simply later. No timing template offers a guarantee of invisibility, and claiming otherwise to a client is a misrepresentation.

The second point to hold onto is that speed and accuracy are in tension, not in agreement. A faster template does not produce more accurate results; it produces fewer, because a reply that arrives after nmap has stopped listening is recorded as though it never arrived. When two runs of the same scan disagree, timing is the usual explanation.

3. Fragmentation and Decoy Scanning

The session examined two techniques whose stated purpose is to make a scan harder to attribute or detect. Both were run, and both deserve an accurate description of what they achieve today.

3.1 Fragmentation

```
sudo nmap -f $TARGET        # fragment packets into 8-byte chunks
sudo nmap -f -f $TARGET     # double fragmentation
sudo nmap --mtu 16 $TARGET  # set an explicit fragment size
```

Fragmentation divides each packet into smaller pieces. The technique dates from an era when intermediary devices — early firewalls and intrusion detection systems — inspected only the first fragment of a packet and discarded or ignored the rest. A scan split across several fragments could therefore pass through such a device without its port number ever being examined. Modern devices reassemble fragments before inspection, which removes the technique's purpose on current infrastructure.

3.2 Decoy scanning

```
# explicit decoy addresses
sudo nmap -D 192.168.50.100,192.168.50.50,192.168.50.200,192.168.50.128 $TARGET

# randomly generated decoys
sudo nmap -D RND:5 $TARGET
```

Decoy scanning sends additional packets from falsified source addresses, so that the target's logs record the scan as arriving from several sources simultaneously. The intent is to prevent the defender from determining which single address was responsible. The explicit form allows you to choose the addresses; the RND form lets nmap generate them.

THE SCOPE CONSEQUENCE There is a scope implication that is easy to overlook. Decoy addresses are real destinations: packets are sent toward them. If a decoy address belongs to a host that is not within your authorised scope, you have sent traffic to an out-of-scope system, and that is an authorisation breach regardless of intent. In a laboratory this is inconsequential. On a client engagement, every decoy address must be listed in the scope document in advance. Many engagement letters prohibit decoys outright.

3.3 The honest assessment

WHAT TO TELL A CLIENT Both techniques should be understood for three reasons: they appear in certification material and interviews, they still function against unmaintained equipment, and they teach packet-level reasoning that is useful everywhere else. What they should not be used for is a sales claim. Modern firewalls, web application firewalls and endpoint detection platforms reassemble fragments and correlate decoys against baseline traffic. Telling a client that your scan was undetectable, on the basis of having used these flags, is an assertion you cannot support.

4. The Nmap Scripting Engine

NSE extends nmap beyond port and version detection into active enumeration and, in some cases, vulnerability checking. The engine is powerful and the community library is large, but it is also the part of nmap most likely to cause harm if used without reading the documentation first.

4.1 Aggressive scanning versus explicit flags

```
nmap -A $TARGET
nmap -sV -sC $TARGET
```

Flag	What it enables	Noise level
-A	Equivalent to -O, -sV, -sC and --traceroute together	High
-sV	Service and version detection	Moderate
-sC	The default category of NSE scripts	Moderate
-O	Operating system fingerprinting	Moderate
--traceroute	The network path to the target	Low

-A is convenient because it performs several operations at once, but it obscures which flag produced which output. While you are learning, run the flags separately so that you can attribute every line of output to a cause. When time is constrained and scope permits, -A is a reasonable choice — provided you can still explain what it did.

4.2 Locating scripts

```
cd /usr/share/nmap/scripts
ls | head
find . -name 'ftp*.nse'
find . -name 'smb*.nse' | head
ls | wc -l
```

```
EXPECTED OUTPUT
./ftp-anon.nse
./ftp-bounce.nse
./ftp-brute.nse
./ftp-libopie.nse
./ftp-proftpd-backdoor.nse
./ftp-syst.nse
./ftp-vsftpd-backdoor.nse
612
```

A Kali installation ships 612 NSE scripts as verified during preparation. Note that the list includes ftp-vsftpd-backdoor.nse, which relates directly to the vulnerability examined later in this handout.

A SMALL SYNTAX POINT WITH A LARGER LESSON The session used the form 'find ftp*.nse' without a search path. It produced results because the shell expanded the wildcard before find ran, and because the working directory happened to be the scripts directory. That is a coincidence of situation, not correct syntax. The reliable form is 'find . -name "ftp*.nse"', which instructs find itself to search the current directory tree. Use the correct form consistently and it will continue to work when your working directory changes.

Before running any script, read its description:

```
nmap --script-help ftp-anon
```

The help output states what the script does, which categories it belongs to, and often what arguments it accepts. A script in the brute or intrusive category can lock accounts or destabilise a service; you should know that before it runs, not after.

4.3 FTP scripts

```
nmap -p21 --script ftp-anon.nse $TARGET
nmap -p21 --script ftp-syst.nse,ftp-bounce.nse $TARGET
```

```
EXPECTED OUTPUT
PORT   STATE SERVICE
21/tcp open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT: FTP server status: vsFTPd 2.3.4
|_  Current time is 09:20:00
```

The anonymous FTP result is a finding in its own right: it means that anyone who can reach port 21 can read whatever the FTP service exposes, without credentials. The ftp-syst result supplies the server software and version, which corroborates the earlier version detection.

```
nmap -p21 --script ftp-brute.nse --script-args userdb=users.txt,passdb=pass.txt $TARGET
```

AUTHORISATION REQUIRED The ftp-brute script performs password guessing. It was demonstrated in the session as syntax, which is appropriate; actually running it against a system requires explicit written permission, because it will trigger account lockout policies and can take the service down. In a laboratory it is harmless. In any other context, running it without authorisation is an attack, not a test.

4.4 HTTP enumeration scripts

```
nmap -p80,8180 $TARGET
nmap -p80,8180 --script http-enum.nse $TARGET
nmap -p80,8180 --script http-title.nse $TARGET
nmap -p80,8180 --script http-methods.nse $TARGET
nmap -p80,8180 --script http-robots.txt.nse $TARGET
```

```
EXPECTED OUTPUT
PORT     STATE SERVICE
80/tcp   open  http
| http-enum:
|   /phpinfo.php: PHP info page
|   /icons/: Icons
|   /tikiwiki/: TikiWiki
|   /dav/: WebDAV
| http-methods:
|   Supported Methods: GET HEAD POST OPTIONS
|_  TRACE: vulnerable
```

CORRECTION: THE ROBOTS SCRIPT NAME A correction that matters in practice. The session attempted the robots script as http-robots.nse, and nmap reported that the name did not match any category, filename or directory. The correct filename is http-robots.txt.nse, which the session then used successfully. This is not a criticism of the attempt; it is an illustration of a rule: script names are exact, and when a name fails, list the directory rather than guessing again. Every name in this handout was verified against an installed copy of the scripts directory.

4.5 SMB scripts

```
find /usr/share/nmap/scripts/ -name 'smb*.nse' | head -10
nmap -p139,445 --script smb-os-discovery,smb-enum-shares $TARGET
nmap -p139,445 --script smb-vuln-ms17-010 $TARGET
smbclient -L //$TARGET -N
```

```
EXPECTED OUTPUT
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   Workgroup: WORKGROUP
| smb-enum-shares:
|   print$
|   tmp
|_  opt
```

SMB deserves particular attention because it has been the entry point for several of the most damaging worm outbreaks on record, and because it is often the service that exposes the most information without requiring credentials. The null session — connecting with no username or password — should not return a share list on a well-configured system. When it does, that is a configuration finding.

5. From a Version Number to a CVE

This section describes the workflow that converts a service banner into a documented, scored and defensible finding. It is the core professional skill of this day.

5.1 The research sequence

`nmap -sV $TARGET  ->  21/tcp open ftp vsftpd 2.3.4`

Step	Action	Where
1	Search for the exact product and version together	General search: "vsftpd 2.3.4 exploit"
2	Identify the CVE identifier	nvd.nist.gov, cve.mitre.org
3	Read the authoritative description	The NVD record for that CVE
4	Check the vendor and distribution advisories	Debian security tracker, Red Hat, Ubuntu
5	Locate a public proof of concept	Exploit-DB, GitHub
6	Read the CVSS score and vector	The NVD record
7	Classify the weakness	cwe.mitre.org
8	Write the finding	Your report

5.2 The identifiers, and what each is for

Identifier	Full name	The question it answers	Example
CVE	Common Vulnerabilities and Exposures	Which specific flaw is this?	CVE-2011-2523
CVSS	Common Vulnerability Scoring System	How severe is it?	9.8 Critical
CWE	Common Weakness Enumeration	What class of weakness is it?	CWE-78
CAPEC	Common Attack Pattern Enumeration	How is it attacked?	CAPEC-88
CPE	Common Platform Enumeration	What precisely is affected?	cpe:/a:vsftpd:vsftpd:2.3.4

KEEP THEM DISTINCT These identifiers answer five different questions, and they are not interchangeable. CVE names the specific instance. CVSS quantifies severity. CWE names the category of mistake. Confusing them produces statements such as 'CWE-2011-2523', which is not a valid identifier of any kind and which immediately signals to a reviewer that the document was assembled without understanding. Keep the three distinct in your own writing.

5.3 CVE-2011-2523 — a complete worked example

Field	Value
CVE identifier	CVE-2011-2523
Affected component	vsftpd 2.3.4, distributed between 30 June and 3 July 2011
Authoritative description	“vsftpd 2.3.4 downloaded between 20110630 and 20110703 contains a backdoor which opens a shell on port 6200/tcp.”
Weakness class	CWE-78 — Improper Neutralization of Special Elements used in an OS Command
CVSS v3.1 base score	9.8 (Critical)
CVSS vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Mechanism	A username containing a specific character sequence triggers code embedded in the distributed archive, opening a shell listener on port 6200
Remediation	Upgrade to a current vsftpd release; verify package signatures from the vendor
References	Debian security tracker; Red Hat CVE database; openwall oss-security, 11 July 2011

The description above is quoted verbatim from the Debian security tracker. That tracker also records a detail of considerable professional interest: the backdoored version was never present in the Debian archive. In other words, the vulnerability did not originate in the software's source code. It was introduced into the distribution archive on the download server, which makes this a supply chain compromise rather than a coding error. That distinction changes the remediation advice: patching the application would not have helped an organisation that had downloaded the compromised archive, because the archive itself was the malicious component.

5.4 Reading the CVSS vector

The base score is a single number, and a single number cannot explain a vulnerability. The vector string is the explanation, and it should be read and understood metric by metric.

Metric	Value	Meaning in plain terms	What it implies
AV — Attack Vector	N (Network)	Reachable over the network; no local access required	An attacker anywhere can attempt it
AC — Attack Complexity	L (Low)	No special conditions; succeeds reliably	Trivial to reproduce
PR — Privileges Required	N (None)	No account is needed	Anonymous exploitation
UI — User Interaction	N (None)	No action from a victim is required	Fully automatable
S — Scope	U (Unchanged)	Impact confined to the vulnerable component	Blast radius limited to that host
C — Confidentiality	H (High)	Full read access to data	All data on the host is exposed
I — Integrity	H (High)	Full write access	Anything can be modified
A — Availability	H (High)	Service can be stopped	The host can be taken offline

WHY THE VECTOR, NOT JUST THE SCORE Reading the vector this way is what allows you to explain a severity rating to a non-technical stakeholder. 'It scores 9.8' means very little to a business owner. 'An attacker anywhere on the network can gain full control of the host with no credentials and no action from any employee' means something immediately. The vector gives you the words for the second sentence.

5.5 Severity bands and the limits of scoring

Score range	Rating	Usual triage decision
0.0	None	No meaningful impact
0.1 – 3.9	Low	Schedule into normal work
4.0 – 6.9	Medium	Plan into a maintenance window
7.0 – 8.9	High	Address within the current cycle
9.0 – 10.0	Critical	Address immediately; assume compromise if unremediated

A caution that belongs in every discussion of CVSS: the base score describes the flaw in isolation. It does not know where that flaw sits in your environment. A critical score on a service that is not reachable from the network the attacker occupies may present less immediate risk than a high-scoring flaw on a public-facing system holding regulated data. Severity informs risk; it does not replace it. The risk framework you encountered in the governance material — likelihood multiplied by impact, in the context of the specific asset — is what turns a score into a decision.

6. Classifying the Weakness with CWE

A CVE identifies one specific instance of a flaw. A CWE identifies the category into which that flaw falls. The distinction matters because remediation happens at the level of the category, not the instance.

CWE	Name	Plain description	Where you encounter it
CWE-78	OS Command Injection	Input reaches a system shell	Backdoors, command injection
CWE-79	Cross-site Scripting	Input reaches HTML without encoding	Web applications
CWE-89	SQL Injection	Input reaches a database query	Web applications, APIs
CWE-22	Path Traversal	Input controls a file path	Download endpoints
CWE-269	Improper Privilege Management	More authority granted than required	Sudo, service accounts, AD
CWE-287	Improper Authentication	Identity is validated incorrectly	Login flows
CWE-306	Missing Authentication	No check performed at all	Administrative interfaces
CWE-798	Use of Hard-coded Credentials	A secret is embedded in code	Repositories, firmware

WHY BOTH BELONG IN THE REPORT Recording CWE-78 alongside CVE-2011-2523 tells a developer something the CVE alone does not: that the underlying mistake is unvalidated input reaching a command interpreter. That knowledge lets

them search their own codebase for the same pattern, rather than patching a single released binary. This is why professional reports carry both identifiers.

7. Writing the Finding

The deliverable of a vulnerability assessment is not a scan output. It is a set of findings, each of which a client can read, verify and act upon. The structure below is the one to use consistently.

7.1 The template

Field	Content
Title	What the issue is, in one short sentence
Severity	CVSS score and rating, with the full vector
Affected asset	Host, port, service and version
Identifiers	CVE, CWE and CPE
Description	The technical description of the flaw
Evidence	The command, its output and a timestamp
Proof of concept	The minimum reproducible steps that demonstrate it
Impact	What an attacker achieves, demonstrated rather than assumed
Remediation	What must change, with a reference
References	Vulnerability databases, vendor advisories, proof-of-concept source

7.2 A complete example

```
TITLE          FTP Backdoor Command Execution in vsftpd 2.3.4
SEVERITY       9.8 Critical   CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
ASSET          192.168.50.129:21/tcp  vsftpd 2.3.4
IDENTIFIERS    CVE-2011-2523 | CWE-78 | cpe:/a:vsftpd:vsftpd:2.3.4

DESCRIPTION    The distributed vsftpd 2.3.4 archive contained a backdoor that
               opens a command shell on port 6200/tcp when a username containing
               a specific character sequence is supplied. The compromised archive
               was available for approximately three days in 2011.

EVIDENCE       nmap -sV -p21 192.168.50.129                 ->  vsftpd 2.3.4
               nmap -p21 --script ftp-vsftpd-backdoor.nse ...  ->  reports VULNERABLE
               Scan performed 25 October 2026, 09:40 +05:00

PROOF OF CONCEPT
 Step 1        Connect to TCP 21 and supply the triggering username sequence.
 Step 2        Confirm that TCP 6200 has opened:  nmap -p6200 192.168.50.129
 Step 3        Connect to TCP 6200 and issue 'id'. The response identifies the
               privilege level of the shell, confirming code execution.

IMPACT         Unauthenticated remote code execution as the FTP service account,
               on a host that also provides database services.

REMEDIATION    Upgrade vsftpd to the current release. Verify vendor signatures.
REFERENCES     security-tracker.debian.org/tracker/CVE-2011-2523
               access.redhat.com/security/cve/CVE-2011-2523
               openwall oss-security, 11 July 2011
```

THE IMPACT STATEMENT IS NOT OPTIONAL Observe the final sentence of the impact statement. The scan established that the host also listens on 3306 (MySQL). Recording that the compromised host also provides database services converts a generic, high-scoring vulnerability into a statement about a specific business exposure. That sentence is the difference between a finding that is filed and a finding that is fixed.

7.3 Why the proof of concept is split into steps

The session divided the demonstration into a first step that establishes the flaw exists and a final step that establishes access. This is deliberate practice, and it has two practical advantages.

1. If the final step cannot be completed — because a tool is unavailable, a version differs, or authorisation is withdrawn — the finding still stands. Step one has already demonstrated that the vulnerable condition is present.

2. A step-by-step proof of concept doubles as a remediation script. The client's administrator can follow the same steps after the fix to confirm that the exposure is closed.

PROFESSIONAL PRACTICE Reports that state only 'exploitation resulted in root access' are of limited use to a client, because they cannot independently verify the result and cannot confirm the fix. The recorded sequence of steps is what makes a finding verifiable in both directions.

8. The Session in Sequence

The following table records the order in which the work was carried out. It is worth memorising, because it is the sequence you will follow on every future assessment.

Phase	Activity	Commands used
1. Orientation	Establish your own address and interfaces	myip, ip a, ifconfig
2. Discovery	Identify live hosts on the subnet	nmap -sn, netdiscover, ipscan
3. Target	Set and verify the target variable	TARGET=..., echo $TARGET
4. Quick scan	Default top-1,000 port scan	nmap $TARGET
5. Full scan	All 65,535 TCP ports	nmap -p- $TARGET
6. Targeted	Confirm specific services	nmap -p21, -p21,22,25,80, -p21-100
7. Fingerprint	OS, UDP and service versions	sudo nmap -O, -sU, -sV
8. Timing study	Observe the speed/accuracy trade-off	nmap -T0, -T5
9. Evasion study	Fragmentation and decoys	nmap -f, nmap -D
10. Scripting	Default scripts, then per-service enumeration	nmap -sV -sC, then NSE
11. Verification	Manual confirmation of script results	ftp, smbclient
12. Research	CVE, CVSS, CWE, proof of concept	NVD, Debian tracker, CWE, Exploit-DB

WHY THIS ORDER Note that manual verification follows automated scanning, not the reverse. The scanner maps the landscape quickly; manual work then confirms whatever appeared interesting. Reversing that order means spending time manually examining services that the scanner would have deprioritised within seconds.

9. Research Assignment

This assignment is the direct preparation for the vulnerability assessment module. Submit as Markdown in the GitHub repository under /s06-scanning.

Task A — compulsory: three complete findings

Select three findings from your own scan of Metasploitable 2 and document each one in full, using the template in section 7.1. Choose findings from different categories where possible.

Category	Candidate from the scan	Why this one is instructive
Remote code execution	vsftpd 2.3.4 — CVE-2011-2523	The complete worked example
Information disclosure	Exposed /phpinfo.php	Requires no exploitation; pure exposure
Misconfiguration	Anonymous FTP or SMB null session	Demonstrates grading a non-CVE finding
Weak protocol	Telnet (23) or rlogin (513)	Cleartext protocol; a design-level finding

Task B — choose any two

Option	Task	Evidence to submit
B1	Run host discovery with nmap, netdiscover and ipscan and explain any difference in results	Three outputs and one explanatory paragraph
B2	Scan the same 100 ports with -T0 and -T5; record timings and compare results	Commands, timings and both outputs
B3	Decode every CVSS metric for CVE-2011-2523 into plain English without consulting NVD's labels	Your own decoded vector
B4	Identify the CWE for three distinct Metasploitable vulnerabilities and explain how remediation differs by class	Three CWE references and your reasoning
B5	Run nmap -f and nmap -D against your own target, capture the traffic, and explain what you observe	tcpdump output and explanation
B6	Write half a page on the vsftpd supply-chain compromise: what happened, how it was discovered, and its modern relevance	Cited sources

Submission requirements

- **Format:** Markdown, in your own words, with screenshots embedded.
- **Evidence:** Every assertion supported by a command, a screenshot or a cited source.
- **Delivery:** Push to GitHub and post the repository link in the cohort channel.

10. Command Reference

Purpose	Command
Set and verify target	TARGET=192.168.50.129 ; echo $TARGET
Host discovery	nmap -sn 192.168.50.0/24
Host discovery (ARP)	sudo netdiscover -r 192.168.50.0/24
Default scan	nmap $TARGET
All ports	nmap -p- $TARGET
Specific port	nmap -p21 $TARGET
Port list	nmap -p21,22,25,80 $TARGET
Port range	nmap -p21-100 $TARGET
Service versions	nmap -sV $TARGET
OS fingerprint	sudo nmap -O $TARGET
UDP scan	nmap -sU $TARGET
Aggressive	nmap -A $TARGET
Default scripts	nmap -sC $TARGET
Timing	nmap -T0 ... -T5
Fragmentation	sudo nmap -f $TARGET

Decoys	sudo nmap -D RND:5 $TARGET
List scripts	cd /usr/share/nmap/scripts && find . -name 'ftp*.nse'
Script description	nmap --script-help ftp-anon
Anonymous FTP check	nmap -p21 --script ftp-anon.nse $TARGET
FTP server status	nmap -p21 --script ftp-syst.nse $TARGET
FTP brute (authorised only)	nmap -p21 --script ftp-brute.nse --script-args userdb=u,p=passdb=p $TARGET
HTTP enumeration	nmap -p80,8180 --script http-enum.nse $TARGET
HTTP title and methods	nmap -p80 --script http-title.nse,http-methods.nse $TARGET
Robots file	nmap -p80 --script http-robots.txt.nse $TARGET
SMB OS and shares	nmap -p139,445 --script smb-os-discovery,smb-enum-shares $TARGET
SMB vulnerability check	nmap -p139,445 --script smb-vuln-ms17-010 $TARGET
SMB share list	smbclient -L //$TARGET -N

Key takeaways

1. The three scanning disciplines are sequential and each depends on the one before.
2. The default nmap scan examines 1,000 ports, not all of them; use -p- when completeness matters.
3. Version detection is what converts a port list into a vulnerability assessment.
4. Faster timing never produces more accurate results, only fewer.
5. CVE identifies the flaw, CVSS quantifies it, CWE classifies it. Keep the three distinct.
6. A finding without a reproducible proof of concept is an opinion.

Next session: the complete vulnerability assessment — consolidating these findings into a prioritised report that a client can act upon.